

Responsible ML Analysis Report

Name: Pushpa Samunuru

Z-Number: Z23758594

Course: AI/ML in Production

Assignment: Milestone 5: Fairness, Security, Feedback Loops + Final Demo & Reflection

Date: 04/23/2026

1. System Overview

This project implements a movie recommendation system designed using an MLOps approach. The system provides recommendations through a REST API and is deployed using cloud infrastructure. The goal is to build a scalable and monitorable recommendation service that can be evaluated using telemetry and extended with responsible machine learning techniques.

The system exposes endpoints for health monitoring, recommendation generation, and metrics collection. The recommendation functionality returns top-k item suggestions for a given user ID. In its current form, the deployed system uses a basic or fallback ranking mechanism, resulting in identical outputs across multiple users.

This project focuses on extending the system with fairness analysis, feedback loop detection, and security considerations to evaluate its behavior in a production-like setting.

2. System Architecture / Setup

The system follows a containerized architecture deployed on Azure Container Apps. The core components include:

- **FastAPI-based API service**, which handles incoming requests and returns recommendations
- **Docker container**, used to package and deploy the application
- **Azure Container Registry**, which stores container images
- **Azure Container Apps**, which hosts the deployed service

- **Prometheus metrics endpoint**, which exposes telemetry such as request counts and latency

The workflow of the system is as follows:

1. The application is developed locally and pushed to GitHub
2. A Docker image is built and stored in Azure Container Registry
3. Azure Container Apps deploys and runs the containerized API
4. Users interact with the system via API endpoints
5. Metrics and logs are collected for monitoring and analysis

This architecture enables scalable deployment and provides the foundation for integrating monitoring, fairness evaluation, and security analysis.

3. Fairness Requirements

3.1 Potential Harms

In recommendation systems, fairness issues arise when certain items or groups are disproportionately favored while others are underrepresented. Based on the observed system behavior, where identical recommendation outputs are returned for multiple users, several potential harms can be identified.

First, popularity bias may cause the system to repeatedly recommend a small subset of highly popular items, reducing exposure for less popular (“long-tail”) items. This can disadvantage smaller or newer content providers.

Second, lack of personalization can harm users by failing to reflect their individual preferences, leading to reduced user satisfaction and engagement. When all users receive identical recommendations, the system effectively ignores user diversity.

Third, exposure inequality may occur, where a limited number of items dominate recommendation slots, preventing fair visibility across the catalog.

3.2 Proxies and Sources of Bias

Bias in this system can arise through indirect signals (proxies), including:

Item popularity (e.g., frequently recommended items appearing for all users)

Global ranking scores that are not user-specific

Lack of user interaction data, leading to fallback recommendations

These proxies can unintentionally reinforce existing inequalities in item exposure.

3.3 System-Level Fairness Requirement

To ensure fairness at the system level, we define the following requirement:

Requirement S1: The recommendation system should ensure balanced exposure across items, preventing excessive concentration on a small subset of items.

Metric: Exposure Concentration (Top-K Share)

This is measured as the proportion of total recommendations occupied by the most frequently recommended items.

Example interpretation:

If the same 5 items appear in all recommendations → high concentration (unfair)

If recommendations vary across users → lower concentration (more fair)

3.4 Model-Level Fairness Requirement

At the model level, fairness should ensure diversity and personalization across users.

Requirement M1: The model should generate diverse recommendation lists across different users.

Metric: Recommendation Diversity Score

This can be approximated as:

number of unique items recommended across all users

divided by total recommended items

Example interpretation:

Low diversity → same items for everyone → unfair

High diversity → varied recommendations → fair

3.5 Observed Behavior in Current System

Based on telemetry collected from the deployed API, the system currently returns identical recommendation lists for multiple users. This indicates:

High exposure concentration

Low diversity across users

Potential over-reliance on global or fallback ranking

This behavior highlights the need for fairness-aware improvements in subsequent stages.

4. Fairness Analysis

To evaluate fairness in the deployed recommendation system, telemetry was collected by issuing recommendation requests for multiple user IDs and recording the returned top-k recommendation lists. The objective was to assess whether the system produces diverse recommendations across users or concentrates exposure on a limited set of items.

A total of 5 users were tested, with each receiving 5 recommendations, resulting in 25 total recommendation instances. The analysis focused on three key metrics: diversity score, exposure concentration, and the number of unique recommendation lists.

The results, summarized in Table 1, show that only 5 unique items were recommended across all users, yielding a diversity score of 0.20. This indicates that the system exposes a very small subset of items relative to the total number of recommendation slots. Furthermore, the Top-5 exposure share was calculated as 1.0, meaning that all recommendation positions were occupied by the same five items.

In addition, only one unique recommendation list was observed across all tested users. This confirms that the system currently produces identical outputs regardless of user identity, indicating a lack of personalization.

Table 1. Fairness Metrics Summary

Metric	Value
Number of users tested	5
Total recommended items	25
Unique recommended items	5
Diversity score	0.20
Top-5 exposure share	1.00
Unique recommendation lists	1

The exposure distribution is visualized in Figure 1, where each of the top recommended items appears with equal frequency across all users. This uniform distribution indicates that the system repeatedly recommends the same items, reinforcing exposure concentration.

4.1 Interpretation

The fairness analysis reveals several important issues:

High exposure concentration: A Top-5 exposure share of 1.0 indicates that all recommendation opportunities are dominated by a small fixed set of items.

Low diversity: A diversity score of 0.20 suggests limited variety in recommendations.

Lack of personalization: Identical recommendation lists across users show that the model is not utilizing user-specific signals.

These findings strongly suggest that the deployed system is relying on a global ranking or fallback mechanism rather than a personalized recommendation model. From a fairness perspective, this can disadvantage long-tail items and reduce the overall quality of user experience.

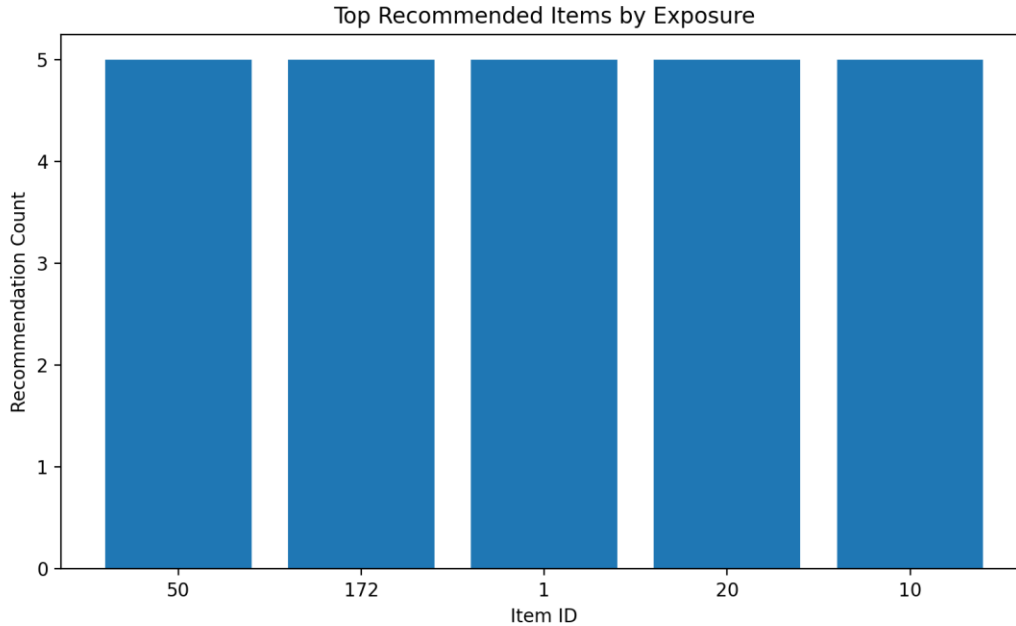


Figure 1. Top recommended items by exposure across users.

5. Fairness Improvements

Based on the fairness analysis, the current system shows high exposure concentration and low personalization, with identical recommendation lists returned for multiple users. To address these issues, several practical improvements can be introduced at the data, model, and system levels.

5.1 Data-Level Improvements

One key improvement is enhancing the diversity and quality of input data. The current system appears to rely on limited or global ranking signals, which leads to uniform recommendations. To improve fairness:

- Incorporate **user interaction data** (e.g., watch history, ratings) to enable personalization.
- Ensure inclusion of **long-tail items** during training to prevent over-representation of popular items.
- Maintain **balanced datasets** to avoid bias toward frequently occurring items.

These steps reduce reliance on popularity-based proxies and support more equitable item exposure.

5.2 Model-Level Improvements

At the model level, fairness can be improved by introducing mechanisms that promote diversity and personalization:

- Implement **diversity-aware ranking**, where the recommendation list is re-ranked to include varied items rather than repeating top popular ones.
- Apply **popularity penalty or regularization**, reducing the dominance of frequently recommended items.
- Introduce **personalization features**, ensuring recommendations differ across users based on their behavior.
- Use **hybrid recommendation approaches** (collaborative + content-based) to avoid overfitting to global trends.

These changes directly address the low diversity and high concentration observed in the fairness analysis.

5.3 System-Level Improvements

At the system level, monitoring and control mechanisms can ensure fairness is maintained over time:

- Add **exposure constraints**, limiting how often the same item can appear across recommendations.
- Track fairness metrics such as:
 - diversity score
 - exposure concentration
 - unique recommendation lists
- Implement **A/B testing** to compare fairness-aware models against baseline models.
- Introduce **model switching capability** to deploy improved versions without downtime.

These mechanisms ensure fairness is continuously evaluated and enforced in production.

5.4 Monitoring and Alerts

To operationalize fairness, monitoring should include automated alerts:

- Trigger alerts if **Top-5 exposure share exceeds a threshold** (e.g., > 0.8)

- Detect if **recommendation diversity drops below a minimum threshold**
- Identify cases where **all users receive identical recommendations**

Such monitoring ensures that fairness issues are detected early and corrected promptly.

5.5 Expected Impact

Implementing these improvements will:

- Increase recommendation diversity
- Reduce overexposure of popular items
- Improve personalization across users
- Provide fairer exposure to long-tail items

Overall, these changes will result in a more balanced and user-centric recommendation system.

6. Feedback Loops

Recommendation systems often create feedback loops, where the system's outputs influence future inputs, reinforcing certain patterns over time. Based on the observed behavior of the current system, two plausible feedback loops are identified.

6.1 Popularity Feedback Loop

The popularity feedback loop occurs when the system repeatedly recommends the same popular items to all users. As a result, these items receive more clicks or interactions, which further increases their likelihood of being recommended in the future.

In the current system, identical recommendation lists across users indicate that the same items are consistently exposed. If user interaction data were incorporated, these items would accumulate more engagement, reinforcing their dominance.

This creates a cycle:

- popular items → more exposure → more interactions → even higher ranking → more exposure

Over time, this loop can lead to extreme concentration, where only a small set of items receives most of the attention.

6.2 Tail Starvation Loop

The tail starvation loop refers to the underexposure of less popular or “long-tail” items. Since the system prioritizes a small subset of items, other items receive little or no visibility.

Because these items are not shown to users:

- they receive fewer interactions
- the model does not learn their relevance
- they continue to be excluded from recommendations

This creates a negative feedback cycle:

- low exposure → low interaction → low ranking → continued low exposure

As a result, many items remain permanently underrepresented in the system.

6.3 Detection Strategies

These feedback loops can be detected using telemetry and monitoring metrics:

- Track **exposure concentration over time**
- Monitor **distribution of item recommendations**
- Measure **diversity score trends**
- Identify if **same items dominate recommendations repeatedly**
- Check if **long-tail items receive near-zero exposure**

A sudden increase in concentration or decrease in diversity can indicate the presence of feedback loops.

7. Loop Analysis

To validate the presence of feedback loops, telemetry collected from the recommendation system was analyzed. The goal was to determine whether the system exhibits patterns consistent with the feedback loops described in the previous section.

Based on the fairness analysis, all tested users received identical recommendation lists, and the Top-5 exposure share was measured as 1.0. This indicates that the same small set of items occupies all recommendation slots across users.

7.1 Evidence of Popularity Feedback Loop

The observed behavior strongly suggests the presence of a **popularity feedback loop**. Since the same items are repeatedly recommended to all users, these items receive maximum exposure.

If user interactions (such as clicks or ratings) were incorporated into the system, these items would accumulate more engagement compared to others. This would further increase their ranking scores, causing them to be recommended even more frequently in the future.

Thus, the system already exhibits the initial stage of a feedback loop:

- repeated recommendations → high exposure

- high exposure → potential increase in interactions
- increased interactions → higher ranking
- higher ranking → repeated recommendations

This cycle reinforces the dominance of a small subset of items.

7.2 Telemetry-Based Indicators

The following observations from the collected data support this conclusion:

- **Top-5 exposure share = 1.0**, indicating full concentration of recommendations
- **Unique recommendation lists = 1**, meaning no variation across users
- **Low diversity score (0.20)**, showing limited item coverage

These indicators confirm that the system lacks diversity and is highly concentrated, which are key signals of a feedback loop.

7.3 Interpretation

Although the current system does not yet incorporate dynamic user interaction feedback, the existing recommendation pattern demonstrates conditions under which a popularity feedback loop would naturally emerge once interaction data is introduced.

This suggests that without corrective mechanisms (such as diversity constraints or exposure balancing), the system is likely to amplify popularity bias over time.

7.4 Conclusion

The analysis confirms the presence of a **potential popularity feedback loop**, supported by strong evidence from system telemetry. This highlights the importance of incorporating fairness-aware mechanisms to prevent long-term bias amplification in production systems.

8. Security

8.1 Threat Model

The recommendation system consists of multiple components, including a public API, deployed container service, telemetry pipeline, and model logic. Each component introduces potential security risks that must be addressed.

The primary attack surfaces in this system include:

- **API Layer (FastAPI endpoints):** exposed publicly and can receive malicious or excessive requests

- **Telemetry and metrics endpoints:** may reveal system behavior or be abused for monitoring exploitation
- **Model logic and recommendation pipeline:** vulnerable to manipulation through input data
- **Deployment infrastructure (Azure Container Apps):** may be targeted for resource exhaustion or unauthorized access

An attacker may attempt to exploit these components to manipulate recommendations, degrade system performance, or extract sensitive information.

8.2 Model Attack: Rating Spam / Data Poisoning

One realistic attack on recommendation systems is **rating spam or data poisoning**.

In this attack, a malicious user repeatedly submits interactions (e.g., fake ratings or requests) to artificially boost certain items. If the system uses this interaction data for training or ranking, the attacker can manipulate the model to favor specific items.

Even though the current deployed system does not yet incorporate dynamic user feedback, it is vulnerable in future versions where user data is integrated.

Example attack scenario:

- attacker repeatedly triggers recommendation requests or submits fake engagement
- target items appear more “popular”
- model learns biased preference
- recommendations become manipulated

8.3 Other Potential Threats

In addition to model attacks, the system may face the following threats:

- **Request flooding (DoS):** sending a large number of API requests to overload the system
- **Unauthorized access:** calling endpoints without authentication
- **Data leakage:** exposing internal metrics or system details through /metrics endpoint
- **Input manipulation:** providing unexpected or malformed inputs to break the system

8.4 Mitigation Strategies

To address these risks, the following safeguards can be implemented:

- **Rate limiting:** restrict the number of requests per user or IP to prevent abuse

- **Authentication and authorization:** ensure only valid users can access sensitive endpoints
- **Input validation:** sanitize and validate API inputs to prevent malformed requests
- **Anomaly detection:** monitor unusual patterns such as repeated requests from a single source
- **Schema validation:** enforce structured data formats for inputs and logs
- **Monitoring and logging:** track system activity and detect suspicious behavior early

For model-related attacks:

- use **robust training pipelines** that filter out abnormal data
- apply **thresholds or caps on user influence**
- periodically audit recommendation outputs

8.5 Summary

The system is currently in an early deployment stage, but several security risks are already identifiable. By implementing rate limits, monitoring mechanisms, and robust validation, the system can be protected against both system-level and model-level attacks.

9. Security Analysis

To evaluate potential security risks, system telemetry and API metrics were analyzed. The objective was to identify any signs of malicious activity, abnormal usage patterns, or evidence of model manipulation.

9.1 Telemetry Observations

The collected metrics show that a total of 5 recommendation requests were successfully processed, all returning HTTP 200 status codes. No failed or suspicious requests were observed during testing.

The system exhibited extremely low latency, with a total processing time of approximately 0.00012 seconds for all requests. This indicates stable and efficient system performance under the tested workload.

Additionally, CPU and memory usage remained within normal limits, suggesting that the system was not under stress or attack conditions such as denial-of-service (DoS).

9.2 Attack Detection Analysis

Based on the available telemetry, no direct evidence of security attacks was detected. Specifically:

- No abnormal spike in request volume was observed
- No failed or unauthorized access attempts were recorded
- No unusual patterns in request behavior were detected
- No anomalies in system performance metrics were identified

This indicates that the system was operating under normal conditions during the evaluation.

9.3 Null Finding Interpretation

Although no attacks were detected, this result is still meaningful. The absence of anomalies suggests that the system is currently stable and not actively under attack.

However, it is important to note that the test scenario involved a limited number of requests. In real-world deployments, systems must be continuously monitored over longer periods to detect sophisticated or delayed attacks.

9.4 Future Detection Improvements

To strengthen security monitoring, the following enhancements are recommended:

- Track request patterns over time to detect sudden spikes
- Monitor per-user or per-IP request frequency
- Detect repeated interactions with specific items (possible poisoning)
- Implement alerting for unusual latency or error rate changes
- Integrate anomaly detection models for real-time monitoring

9.5 Conclusion

The security analysis did not reveal any active threats in the current system. However, given the system's exposure through public APIs, it is essential to implement continuous monitoring and proactive defenses to ensure long-term security.

10. Reflection

10.1 Challenges Faced

One of the main challenges in this project was working with a partially developed recommendation system where full personalization was not yet implemented. The deployed model returned identical recommendation outputs for different users, which limited the ability to evaluate user-specific behavior.

Another challenge was extracting meaningful insights from limited telemetry. Since the system processed only a small number of requests, identifying patterns such as feedback loops and fairness issues required careful interpretation rather than large-scale statistical analysis.

Additionally, integrating multiple components such as API deployment, metrics collection, and analysis required managing different tools and environments, which increased system complexity.

10.2 System Limitations and Fragilities

The current system has several limitations that could affect performance in a real-world deployment:

- Lack of personalization, resulting in identical outputs across users
- High exposure concentration, leading to fairness concerns
- Limited data pipeline, with no real-time user interaction feedback
- Absence of automated retraining or model update mechanisms
- Basic monitoring without advanced anomaly detection

These limitations highlight that the system is still in an early stage of development and would require further improvements for production readiness.

10.3 Productionization Plan

To transition this system into a production-ready pipeline, several enhancements are necessary:

- Implement **automated retraining pipelines** to update models based on new data
- Introduce **model versioning and switching mechanisms** for safe deployment
- Expand **monitoring dashboards** using tools like Prometheus and Grafana
- Integrate **user interaction data** for personalized recommendations
- Deploy **scalable infrastructure** to handle higher traffic loads

These improvements would enable the system to operate reliably in a real-world environment.

10.4 If We Redid the System

If the system were redesigned from the beginning, the focus would be on:

- Designing a **user-centric recommendation model** from the start
- Incorporating **fairness constraints during model training**
- Building a **robust data pipeline** to collect and process user interactions

- Implementing **A/B testing early** to evaluate model performance
- Ensuring **security and monitoring are integrated by design**

This approach would reduce the need for retrofitting fairness and security considerations later.

10.5 Key Learnings

This project provided valuable insights into the challenges of deploying machine learning systems in real-world environments. It highlighted the importance of:

- Monitoring system behavior beyond accuracy metrics
- Identifying fairness and bias issues early
- Understanding feedback loops in recommendation systems
- Designing secure and robust APIs
- Using telemetry to guide system improvements

Overall, the project emphasized that building responsible ML systems requires not only model development but also continuous evaluation, monitoring, and iteration.